Konzeption und Evaluation eines domänenbasierten Explorers als VS-Code Extension für SAP CAP Anwendungen

Bachelorarbeit

aus dem Studiengang Wirtschaftsinformatik Software Engineering

an der Dualen Hochschule Baden-Württemberg Mannheim

von

Max Christian Meinel

05.03.2026

Matrikelnummer, Kurs:	7864687, WWI23SEB
Unternehmen:	SAP SE, 69190, Walldorf
Abteilung:	CAP Tools & MTX
Leiter des Studiengangs:	Prof. Dr. Henning Pagnia
Betreuer im Unternehmen:	Christian Fuchs
Betreuer an der DHBW:	Ulrich Wolf

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Konzeption und Evaluation eines domänenbasierten Explorers als VS-Code Extension für SAP CAP Anwendungen

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Walldorf 05.03.2026

Max Christian Meinel

Abstract

Abstract hier

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Motivation 1
1.2 Problemstellung 1
1.3 Zielsetzung der Arbeit 1
1.4 Forschungsfragen 1
1.5 Aufbau der Arbeit 1

2 Grundlagen **2**
2.1 SAP Cloud Application Programming Model (CAP) 2
2.2 Language Server und Language Server Protocol 2
2.3 VS Code Extension Architektur 2

3 Methodik **3**
3.1 Design Science Research 3
3.2 Einordnung der Arbeit in den DSR-Prozess 3

4 Problem- und Anforderungsanalyse **5**
4.1 Problemkontext und Analyse von CAP-Repositories 5
4.2 Identifizierte Herausforderungen bei der Navigation 9
4.3 Ableitung der Designziele 9
4.4 Funktionale Anforderungen 11
4.5 Abgrenzung der Arbeit 12

5 Integrationsstrategien **13**
5.1 Repository-basierte Integration 13
5.2 CSN-basierte Integration 13
5.3 Language-Server-basierte Integration 14
5.4 Vergleich der Integrationsstrategien 15
5.5 Auswahl der umgesetzten Strategie 15

6 Implementierung des Prototyps **16**
6.1 Technische Architektur der Extension 16
6.2 Umsetzung der Integrationsansätze 16
6.3 Visualisierung im VS Code Explorer 16
6.4 Update- und Synchronisationsstrategie 16

7 Evaluation **17**

7.1 Evaluationskriterien 17
7.2 Vergleich der Integrationsansätze 17
7.3 Analyse der Ergebnisse 17

8 Diskussion 18
8.1 Interpretation der Ergebnisse 18
8.2 Limitationen 18

9 Fazit und Ausblick 19
9.1 Zusammenfassung der Ergebnisse 19
9.2 Beantwortung der Forschungsfragen 19
9.3 Ausblick auf zukünftige Arbeiten 19

Literatur ..... a

Abbildungsverzeichnis

Abbildung 1 Konzeptionelle Fragmentierung eines CAP-Services (vereinfacht). 7

Tabellenverzeichnis

Tabelle 1 Zentrale Artefakte eines CAP-Services **8**

Tabelle 2 Vergleich der Integrationsstrategien anhand definierter Bewertungskriterien **15**

Codeverzeichnis

Abbildung 1 Exemplarische Struktur eines SAP CAP-Projekts (vereinfacht) [1]. 6

1 Einleitung

1.1 Motivation

1.2 Problemstellung

1.3 Zielsetzung der Arbeit

1.4 Forschungsfragen

1.5 Aufbau der Arbeit

2 Grundlagen

2.1 SAP Cloud Application Programming Model (CAP)

2.1.1 CDS und CDL

2.1.2 Core Schema Notation (CSN)

2.2 Language Server und Language Server Protocol

2.2.1 Architektur des Language Servers

2.2.2 Relevante LSP-Funktionalitäten

2.3 VS Code Extension Architektur

2.3.1 Tree View API

2.3.2 Integration von Language Servern

3 Methodik

3.1 Design Science Research

3.2 Einordnung der Arbeit in den DSR-Prozess

3.2.1 Problemidentifikation

3.2.2 Zieldefinition

3.2.3 Artefaktentwurf

3.2.4 Demonstration

3.2.5 Evaluation

Für diese Arbeit wird der Design-Science-Research-Ansatz (DSR) als methodische Grundlage verwendet. Dieser Ansatz eignet sich insbesondere für Arbeiten, bei denen ein konkretes Artefakt entwickelt wird, das ein praktisches Problem adressiert. In dieser Arbeit stellt die entwickelte VS-Code Extension sowie das zugrunde liegende Strukturmodell (domänenbasierter Explorer) dieses Artefakt dar.

Der Design-Science-Research-Ansatz gliedert sich in mehrere Phasen. Zunächst wird das zugrundeliegende Problem präzise analysiert und abgegrenzt. Dabei wird untersucht, welche strukturellen Herausforderungen in größeren SAP CAP Projekten entstehen und weshalb bestehende Werkzeuge nur eingeschränkt eine domänenorientierte Übersicht ermöglichen.

Auf Basis dieser Problemdefinition werden funktionale sowie nicht-funktionale Anforderungen an die Erweiterung definiert. Dazu gehören unter anderem Anforderungen an die Strukturierung der Services, die Filter- und Sortiermöglichkeiten sowie Aspekte der Usability.

Im Anschluss erfolgt die Konzeption des Artefakts. Dabei werden unterschiedliche Modellierungsansätze zur Strukturierung des Repositories betrachtet. Insbesondere werden das Core Schema Notation (CSN) als statische Strukturquelle sowie der Language Server als Quelle für positions- und dateibezogene Informationen analysiert. Ziel ist es, ein geeignetes Modell zu entwerfen, das beide Informationsquellen sinnvoll kombiniert.

Parallel zur konzeptionellen Modellierung werden verschiedene grafische Mockups erstellt. Diese dienen dazu, unterschiedliche Ansätze der Visualisierung und Interaktion zu vergleichen. Für den Vergleich werden zuvor definierte Kriterien herangezogen, beispielsweise Übersichtlichkeit, Nachvollziehbarkeit der Domänenstruktur und Erweiterbarkeit.

Auf Grundlage des ausgewählten Konzepts wird anschließend ein prototypischer Explorer als VS-Code Extension implementiert. Die Implementierung dient dabei nicht primär als Produktivlösung, sondern als Demonstrator des entworfenen Modells.

Abschließend erfolgt eine Evaluation des entwickelten Artefakts. Dabei werden sowohl die strukturelle Modellierung (Kombination von CSN und Language Server) als auch die gewählte grafische Darstellung anhand der zuvor definierten Kriterien bewertet. Ziel der Evaluation ist es, die Eignung des entwickelten Konzepts zur Unterstützung von Entwicklern in größeren CAP-Projekten zu überprüfen.

4 Problem- und Anforderungsanalyse

Dieses Kapitel konkretisiert die im Methodikteil beschriebene Problemidentifikation und Zieldefinition im Kontext von SAP CAP-Projekten. Zunächst wird die strukturelle Organisation typischer CAP-Repositories analysiert und die Verteilung servicebezogener Artefakte untersucht. Darauf aufbauend werden navigationsbezogene Herausforderungen abgeleitet, aus denen Designziele sowie funktionale und nicht-funktionale Anforderungen an die zu entwickelnde VS-Code Extension formuliert werden. Abschließend wird die Arbeit inhaltlich abgegrenzt.

4.1 Problemkontext und Analyse von CAP-Repositories

4.1.1 Typische Struktur von CAP-Projekten

- cap gibt eine strenge struktur vor
- ziel ist es damit SoC, was bei skalierenden Projekten hilft (quelle SoC)
- ein typisches cap Projekt hat folgende Struktur (siehe Abbildung)

```
example-cap-project
├─ db
│  ├─ schema.cds
│  └─ data
│     └─ seed.csv
├─ srv
│  ├─ catalog-service.cds
│  ├─ catalog-service.js
│  ├─ handlers
│  │  └─ validation.js
│  └─ auth
│     └─ roles.cds
├─ app
│  └─ fiori-app
│     └─ manifest.json
├─ test
│  └─ catalog-service.test.js
├─ config
│  └─ cds.env
└─ package.json
```

Abbildung 1 — Exemplarische Struktur eines SAP CAP-Projekts (vereinfacht) [1].

example-cap-project/

- Rootverzeichnis des CAP-Projekts
- Bündelt alle fachlichen, technischen und konfigurativen Artefakte
- Struktur folgt dem Prinzip der Separation of Concerns

db/

- Enthält das fachliche Domänenmodell
- Definition von Entities und Beziehungen in CDS
- Repräsentiert die Persistenz- und Datenstrukturebene

srv/

- Enthält Service-Definitionen und Geschäftslogik
- Exponiert Teile des Domänenmodells als Services
- Verknüpft deklaratives Modell mit imperativer Implementierung

app/

- Enthält optionale Frontend-Anwendungen
- Konsumiert die definierten Services
- Repräsentiert die Präsentationsebene

test/

- Enthält Testfälle zur Validierung der Service-Logik
- Unterstützt Qualitätssicherung und Wartbarkeit

config/

- Enthält umgebungsspezifische Konfigurationen
- Steuerung von Laufzeit- und Deployment-Parametern

package.json

- Projektmetadaten und Abhängigkeiten
- Definition von Skripten für Build, Start und Tests

4.1.2 Verteilung servicebezogener Artefakte

- ein Service setzt sich aus mehreren funktionalen Bereichen zusammen
- diese Bereiche sind unterschiedlichen Verantwortlichkeiten zugeordnet
- die Fragmentierung erfolgt entlang technischer und fachlicher Concerns
- die Zuordnung ist implizit und verteilt

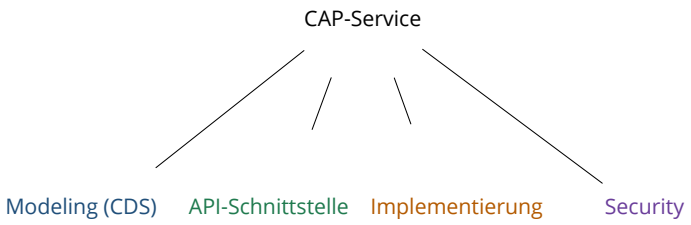

Abbildung 1 — Konzeptionelle Fragmentierung eines CAP-Services (vereinfacht).

Die Abbildung verdeutlicht, dass ein CAP-Service nicht als monolithisches Artefakt vorliegt, sondern sich über mehrere konzeptionelle Ebenen erstreckt.

- Modell- und Service-Definition sind konzeptionell getrennt
- Die Service-Schnittstelle bildet nicht zwingend das vollständige Domänenmodell ab
- Laufzeitverhalten ist nicht im Modell selbst definiert, sondern in separaten Implementierungen
- Sicherheitsrelevante Regeln sind als Annotationen außerhalb der Kernlogik verortet
- Fachliche und technische Aspekte sind strukturell entkoppelt

Kind	Ebene	Beispiele
Entity	Modeling (CDS)	Books, Authors
Type	Modeling (CDS)	UUID, String, Decimal
Association	Modeling (CDS)	to Author, to Items
Aspect	Modeling (CDS)	Reusable model extensions
Service	API-Schnittstelle	CatalogService
Projection	API-Schnittstelle	Books as projection on ...
Action / Function	API-Schnittstelle	submitOrder(), topSelling()
Event Handler	Implementierung	before / on / after
Authorization Annotations	Security	@restrict, @requires

Tabelle 1 — Zentrale Artefakte eines CAP-Services

Tabelle X konkretisiert die in der Abbildung dargestellten Ebenen durch die zentralen Artefakt-Kinds des CAP-Modells.

- Ein vollständiges Serviceverständnis erfordert die Betrachtung mehrerer Artefakt-Kategorien
- Die Beziehungen zwischen den Artefakten werden über Referenzen hergestellt
- Eine zentrale, aggregierte Repräsentation eines Services existiert nicht auf Artefaktebene
- Änderungen an einem Service können mehrere Ebenen gleichzeitig betreffen
- Die Service-Identität ergibt sich erst aus dem Zusammenspiel aller Ebenen

Wichtig: CAP orientiert sich an DDD und an seperation of concerns! -> https://cap.cloud.sap/docs/guides/domain/#domain-entities -> SoC

4.2 Identifizierte Herausforderungen bei der Navigation

4.2.1 Verteilte Artefakte und implizite Abhängigkeiten

- Ein Service erstreckt sich über mehrere konzeptionelle Ebenen
- Modell, Schnittstelle, Implementierung und Sicherheitsregeln sind physisch getrennt
- Die Beziehungen zwischen diesen Artefakten sind referenziell, nicht visuell explizit
- Projektionen referenzieren Entities
- Event Handler referenzieren Service-Operationen
- Sicherheitsannotation beeinflussen Verhalten ohne zentrale Sichtbarkeit
- Kein Artefakt enthält die vollständige Service-Definition

4.2.2 Erhöhter Navigationsaufwand und fehlende Service-Übersicht

- Zur Analyse eines Services sind mehrere Dateiwechsel erforderlich
- Wechsel zwischen deklarativer Modellbeschreibung und imperativer Implementierung
- Sicherheitsregeln müssen separat geprüft werden
- Die Service-Identität ergibt sich erst durch manuelle Rekonstruktion
- Es existiert keine aggregierte Sicht auf alle servicebezogenen Artefakte
- Kontextwechsel erhöhen kognitive Belastung
- Änderungen an einem Service betreffen potenziell mehrere Ebenen

4.3 Ableitung der Designziele

- auf Basis der identifizierten Navigationsprobleme werden zentrale Designziele abgeleitet

- Ziel ist die Definition lösungsneutraler Anforderungen an ein unterstützendes Artefakt
- die Designziele konkretisieren, welche Eigenschaften eine geeignete Lösung aufweisen muss
- sie bilden die Grundlage für die spätere Anforderungsdefinition und Artefaktgestaltung

Aggregierte Service-Sicht

- servicebezogene Artefakte sollen konsolidiert dargestellt werden
- eine einheitliche Einstiegsperspektive auf einen Service soll ermöglicht werden
- Modell, Schnittstelle, Implementierung und Sicherheitsaspekte sollen gemeinsam sichtbar sein
- die Service-Identität soll ohne manuelle Rekonstruktion erfassbar sein

Reduktion von Navigations- und Kontextwechseln

- die Anzahl notwendiger Dateiwechsel soll minimiert werden
- deklarative und imperative Artefakte sollen kontextnah zugänglich sein
- relevante Artefakte sollen schnell auffindbar sein
- kognitive Kontextwechsel zwischen Ebenen sollen reduziert werden

Explizite Darstellung von Abhängigkeiten

- Beziehungen zwischen Entities, Projektionen und Operationen sollen sichtbar gemacht werden
- Verknüpfungen zwischen Service-Definition und Implementierung sollen nachvollziehbar sein
- sicherheitsrelevante Annotationen sollen eindeutig zuordenbar sein
- implizite Referenzen sollen transparent werden

Servicezentrierte Navigation

- die Navigation soll sich an fachlichen Services orientieren
- die technische Ordnerstruktur soll nicht primärer Navigationsanker sein
- der Service soll als logische Einheit im Mittelpunkt stehen
- eine strukturierte Darstellung entlang konzeptioneller Ebenen soll ermöglicht werden

4.4 Funktionale Anforderungen

Aus den zuvor formulierten Designzielen lassen sich konkrete funktionale Anforderungen an die zu entwickelnde VS-Code Extension ableiten. Während die Designziele die gewünschten Eigenschaften der Lösung auf konzeptioneller Ebene beschreiben, spezifizieren die folgenden Anforderungen die Funktionen, die das Artefakt zur Unterstützung der Navigation in CAP-Projekten bereitstellen muss.

FA1 Identifikation von CAP-Services: Die Extension muss CAP-Services innerhalb eines Projekts automatisch identifizieren.

FA2 Aggregierte Serviceübersicht: Die Extension muss eine aggregierte Darstellung aller zu einem Service gehörenden Artefakte bereitstellen.

FA3 Navigation zu Artefakten: Die Extension muss eine direkte Navigation von der Serviceübersicht zu den entsprechenden Artefakten im Quellcode ermöglichen.

FA4 Darstellung von Artefaktbeziehungen: Die Extension muss Beziehungen zwischen servicebezogenen Artefakten sichtbar machen, insbesondere zwischen Service-Definitionen, Projektionen, Entities und zugehörigen Implementierungen.

4.4.1 Nicht-funktionale Anforderungen

Neben den funktionalen Anforderungen ergeben sich aus dem Nutzungskontext innerhalb der Entwicklungsumgebung zusätzliche nicht-funktionale Anforderungen. Diese beschreiben qualitative Eigenschaften und Rahmenbedingungen, die bei der Umsetzung der Extension berücksichtigt werden müssen.

NFA1 Integration in Visual Studio Code: Die Lösung muss als Erweiterung für die Entwicklungsumgebung Visual Studio Code implementiert werden und sich in deren Erweiterungsmechanismus integrieren.

NFA2 Kompatibilität mit CAP-Projektstrukturen: Die Extension muss mit der typischen Struktur von SAP CAP-Projekten kompatibel sein und relevante Artefakte in den vorgesehenen Verzeichnissen erkennen können.

NFA3 Performante Analyse von Projekten: Die Analyse eines CAP-Repositories soll ohne merkliche Verzögerung erfolgen, sodass die Nutzung der Extension den Entwicklungsworkflow nicht beeinträchtigt.

NFA4 Aktualität der Artefaktübersicht: Änderungen an relevanten Projektartefakten müssen automatisch erkannt werden, sodass die dargestellte Serviceübersicht den aktuellen Projektzustand widerspiegelt.

4.5 Abgrenzung der Arbeit

Die vorliegende Arbeit fokussiert auf die Unterstützung der Navigation in CAP-Projekten durch eine VS-Code Extension. Daraus ergeben sich mehrere bewusste Einschränkungen des Untersuchungs- und Lösungsumfangs.

- Die Extension basiert ausschließlich auf der statischen Analyse von Projektartefakten. Eine Analyse des Laufzeitverhaltens von CAP-Services erfolgt nicht.
- Die Lösung orientiert sich an der typischen Struktur von CAP-Projekten.
- Ziel der Extension ist die Verbesserung der Navigation zwischen servicebezogenen Artefakten. Funktionen zur automatischen Codegenerierung oder Refaktorierung sind nicht Bestandteil der Lösung.
- Die Evaluation der Lösung erfolgt anhand ausgewählter CAP-Projekte; eine umfassende Nutzerstudie mit CAP-Entwicklern ist nicht Bestandteil der Arbeit.

5 Integrationsstrategien

5.1 Repository-basierte Integration

- die Repository-basierte Integration rekonstruiert die Service-Struktur direkt aus den Dateien des Projektrepositories
- die Analyse erfolgt auf Grundlage der vorhandenen Projektstruktur sowie der enthaltenen CDS- und Implementierungsdateien
- relevante Artefakte werden durch Parsing der .cds Dateien identifiziert
- Entities, Services, Projektionen sowie Operationen können auf diese Weise extrahiert werden
- Implementierungslogik wird durch Analyse von .js oder .ts Dateien ermittelt
- Beziehungen zwischen Artefakten müssen aus Referenzen im Code rekonstruiert werden
- dieser Ansatz arbeitet primär auf syntaktischer Ebene der Projektartefakte
- dadurch entsteht ein hoher Implementierungsaufwand für Parsing und Referenzauflösung
- zusätzlich besteht das Risiko, dass semantische Aspekte der CDS-Sprache nicht vollständig korrekt interpretiert werden
- gleichzeitig werden Analysefunktionen potenziell dupliziert, die bereits durch bestehende CAP-Tooling-Komponenten bereitgestellt werden

5.2 CSN-basierte Integration

- die CSN-basierte Integration nutzt das von CAP generierte Core Schema Notation (CSN) Modell als Informationsquelle
- das CSN-Modell stellt eine kompilierte Repräsentation aller CDS-Artefakte eines Projekts dar

- Entities, Services, Projektionen sowie Operationen sind im Modell strukturiert enthalten
- Beziehungen zwischen Artefakten sind bereits im Modell explizit aufgelöst
- die Service-Struktur kann dadurch direkt aus der Modellrepräsentation abgeleitet werden
- im Vergleich zur repository-basierten Analyse entfällt das manuelle Parsing der CDS-Dateien
- dadurch wird eine höhere semantische Genauigkeit der Artefaktstruktur erreicht
- jedoch enthält das CSN-Modell keine vollständigen Informationen über Implementierungsartefakte
- insbesondere fehlen konkrete Positionsinformationen innerhalb der Quelltexte
- für eine Navigation im Editor müssen diese Informationen daher separat ermittelt werden

5.3 Language-Server-basierte Integration

- die Language-Server-basierte Integration nutzt Informationen des CAP Language Servers als zentrale Analysequelle
- der Language Server stellt bereits semantische Analysen der CDS-Artefakte innerhalb der Entwicklungsumgebung bereit
- über die bereitgestellten Funktionen können Services, Entities sowie weitere Modellartefakte identifiziert werden
- zusätzlich stehen Positionsinformationen zu Definitionen und Referenzen im Quellcode zur Verfügung
- dadurch können Artefakte direkt mit ihren entsprechenden Codepositionen verknüpft werden
- im Gegensatz zur repository-basierten Analyse entfällt das eigene Parsing der CDS-Dateien
- auch gegenüber der CSN-basierten Integration bietet dieser Ansatz direkten Zugriff auf editorbezogene Informationen

- die Strategie nutzt damit bestehende Analysefunktionen der Entwicklungsumgebung
- dies reduziert den Implementierungsaufwand und vermeidet redundante Analyseprozesse
- gleichzeitig entsteht eine Abhängigkeit von den bereitgestellten Funktionen des Language Servers

5.4 Vergleich der Integrationsstrategien

- zur Bewertung der Integrationsstrategien werden mehrere Kriterien herangezogen
- diese Kriterien leiten sich aus den zuvor definierten Designzielen ab
- sie betreffen insbesondere die semantische Genauigkeit der Analyse, den Integrationsgrad in die Entwicklungsumgebung sowie den Implementierungs- und Wartungsaufwand
- auf dieser Grundlage werden die drei Strategien vergleichend gegenübergestellt

Kriterium	Repository	CSN	Language Server
Implementierungsaufwand	hoch	mittel	mittel
Semantische Genauigkeit	gering	hoch	hoch
Positionsinformationen im Code	vorhanden	begrenzt	präzise
Integrationsgrad in VS Code	gering	mittel	hoch
Wartbarkeit	gering	mittel	hoch
Abhängigkeit von CAP-Tooling	gering	hoch	hoch

Tabelle 2 — Vergleich der Integrationsstrategien anhand definierter Bewertungskriterien

5.5 Auswahl der umgesetzten Strategie

6 Implementierung des Prototyps

6.1 Technische Architektur der Extension

6.2 Umsetzung der Integrationsansätze

6.3 Visualisierung im VS Code Explorer

6.4 Update- und Synchronisationsstrategie

7 Evaluation

7.1 Evaluationskriterien

7.2 Vergleich der Integrationsansätze

7.3 Analyse der Ergebnisse

8 Diskussion

8.1 Interpretation der Ergebnisse

8.2 Limitationen

9 Fazit und Ausblick

9.1 Zusammenfassung der Ergebnisse

9.2 Beantwortung der Forschungsfragen

9.3 Ausblick auf zukünftige Arbeiten

Literatur

[1] capire, „GitHub - capire/bookshop: Our primer sample to get started in a nutshell“. Zugegriffen: 25. Februar 2026. [Online]. Verfügbar unter: https://github.com/capire/bookshop